caseSetup-v4.2 Technical User Guide

S. Chen

Jun 2026

This document covers the use of caseSetup for external automotive aerodynamics cases.

Contents

1 Introduction

caseSetup was built over a couple years for the Ontario Tech Racing team. This process is a mix of experiences gained from FSAE related activities and professional activities. The first iteration of caseSetup was born in the fall of 2022. The drive behind caseSetup was to improve the consistency of runs from user to user while reducing the chances of mistakes. The main idea behind it was to use pre-defined templates which mildly restricts the user from making un-intended modifications to the setup. This allows for consistent meshing, solving, and post processing regardless of user.

This document here is the technical user guide, while some Computational Fluid Dynamics (CFD) theories may be included, it is not meant to be an exhaustive explination or discussion of the physics. Please refer to specialized CFD books and resources if more information is required.

2 Directory Structure

This process requires a specific directory structure for your cases to work. It looks like the following:

- JOB-DIRECTORY
  - 01_incoming(YYYY-MM-DD)
    - 2025-09-11_New_Geometry
  - 02_references
    - ANSA
      - trial009.ansa.gz
    - MSH
      - trial009.obj.gz
      - fw-v1-1.obj.gz
  - 03_reports
    - 051_064_063_report_2025-11-16.pptx
  - CASES
    - 001
    - 002
    - 003
    - ...
    - 009
    - **caseSetup**
      - constant
      - system

The definitions for each folder is described here:

- 01_incoming(YYYY-MM-DD) - Contains all the incoming geometry from other departments or other files which are required for the simulation. Each instance of CAD delivered to the department must be in a dated folder with a short descriptive name.
- 02_references - Contains all the geometry files that are used in the simulation. Sub-directories are ANSA and MSH
- ANSA - Contains all the ANSA files that are used in the simulation, ideally contains the CAD files within them and not just the tri-surfaces. Should be saved as ".gz" for smaller file size.
- MSH - Contains all the tri-surfaces that are used in the simulation. Ideally this is where the original copies of all the STLs or OBJ files live. Each trial folder contains only a symbolic link referencing to this directory to reduce the number and size of geometry files.

- `03_reports` - Contains all the reports placed by the `pptGeneration` process or placed by the user manually.
- `CASES` - Contains all the simulation runs, this is where each case/trial folder lives, each trial is denoted by a 3 digit number XXX.
- `caseSetup` - This is the file that controls the simulations and the outputs.
- `system` - This directory contains the files required for the simulation to run.
- `constant` - Like `system` this directory also contains the files required for the simulation and also in `constant/triSurfaces` is where the geometry for the simulation is stored for this case. It is ideally a symbolic link so that repeated geometries are not copied many times in many cases.

Once all these directories are setup, we are ready to explore the rest of the features.

3 Theory

This section gives the theoretical background needed to interpret the turbulence models selected by `caseSetup`. The equations below use the incompressible, constant-density form for clarity. The same modelling ideas apply to the compressible formulations, with the additional energy equation and density variations.

3.1 Reynolds-Averaged Navier–Stokes (RANS)

In a turbulent flow, each instantaneous quantity is decomposed into a mean and a fluctuating component,

$$u_i = \overline{u_i} + u_i', \qquad p = \overline{p} + p'. \tag{1}$$

The averaging operation removes the rapidly varying fluctuations. Applying it to the incompressible continuity and momentum equations gives

$$\frac{\partial \overline{u_i}}{\partial x_i} = 0, \tag{2}$$

$$\frac{\partial \overline{u_i}}{\partial t} + \overline{u_j}\frac{\partial \overline{u_i}}{\partial x_j} = -\frac{1}{\rho}\frac{\partial \overline{p}}{\partial x_i} + \nu\frac{\partial^2 \overline{u_i}}{\partial x_j \partial x_j} - \frac{\partial \overline{u_i' u_j'}}{\partial x_j}. \tag{3}$$

The final term is the divergence of the Reynolds-stress tensor. It represents the transport of momentum by turbulent fluctuations and introduces more unknowns than equations; this is the RANS closure problem.

For most two-equation and one-equation models, the Reynolds stresses are approximated with the Boussinesq hypothesis,

$$-\rho\overline{u_i' u_j'} = 2\mu_t S_{ij} - \frac{2}{3}\rho k \delta_{ij}, \qquad S_{ij} = \frac{1}{2}\left(\frac{\partial \overline{u_i}}{\partial x_j} + \frac{\partial \overline{u_j}}{\partial x_i}\right), \tag{4}$$

where μ_t is the turbulent (eddy) viscosity and

$$k = \frac{1}{2}\overline{u_i' u_i'} \tag{5}$$

is the turbulent kinetic energy. The practical effect is that turbulence is modelled as an additional momentum diffusion, with μ_t obtained from one or more transport equations.

RANS is appropriate when the engineering objective is a statistically steady solution, such as mean forces, mean pressure distributions and integrated aerodynamic coefficients. It does not resolve the full turbulent spectrum. Instead, the selected model represents the average influence of turbulence on the resolved flow. Mesh quality, wall treatment, inlet turbulence quantities and convergence therefore affect the accuracy of the result as much as the nominal model choice.

3.2 Turbulence-model theory for RANS

3.2.1 k–epsilon models

The standard k–ε family solves transport equations for k and the dissipation rate ε. The turbulent viscosity is obtained from

$$\mu_t = \rho C_\mu \frac{k^2}{\varepsilon}. \tag{6}$$

These models are robust and commonly perform well in free-shear flows, mixing layers and fully turbulent regions. They are less reliable in strongly separated flows, adverse pressure gradients and near-wall regions unless an appropriate wall treatment and model variant are used. The repository contains a `kEpsilon` field template, but it is not currently exposed as a selectable `TURB_MODEL` in the case-generation mapping.

3.2.2 k–omega models and SST

The k–ω family solves for turbulent kinetic energy k and specific dissipation rate ω. A representative eddy-viscosity relation is

$$\mu_t \sim \rho \frac{k}{\omega}. \tag{7}$$

The ω variable gives the model useful near-wall behaviour, while the formulation can be sensitive to free-stream values. The Shear Stress Transport model, k–ω SST, blends a near-wall k–ω formulation with a more free-stream-robust k–ε behaviour and limits the eddy viscosity in regions of strong strain. This makes SST a common choice for external aerodynamics and separated flows. In `caseSetup`, the steady alias `kosst` selects `kOmegaSST`.

3.2.3 Spalart–Allmaras

Spalart–Allmaras is a one-equation model developed primarily for aerodynamic boundary layers. It transports a modified turbulent-viscosity variable, commonly written $\widetilde{\nu}$, and obtains the eddy viscosity through a damping function,

$$\mu_t = \rho \widetilde{\nu} f_{v1}. \tag{8}$$

Its low computational cost and good performance for attached boundary layers make it useful for vehicle aerodynamics. It can be less dependable for complex separation, massive recirculation and strongly three-dimensional vortical flows. The steady alias `sa` selects `SpalartAllmaras`.

3.2.4 GEKO

GEKO (Generalized k–ω) is a coefficient-tunable two-equation model. It retains the general k–ω structure while allowing calibrated coefficients to alter the response to separation, jets, wakes and free shear. Its results are therefore dependent on both the implementation and the selected coefficient set. The steady alias `geko` selects `GEKO`; users should record any GEKO coefficient changes with the case because they directly affect model behaviour.

3.3 Wall-modelling theory

The near-wall region contains a viscous sublayer, a buffer layer and an outer logarithmic layer. The dimensionless wall distance is

$$y^+ = \frac{u_\tau y}{\nu}, \qquad u_\tau = \sqrt{\frac{\tau_w}{\rho}}, \tag{9}$$

where y is the distance from the wall, τ_w is wall shear stress and u_τ is friction velocity. In the viscous sublayer, the mean velocity approximately follows $u^+ = y^+$. Farther from the wall, the log-law approximation is commonly written

$$u^+ = \frac{1}{\kappa} \ln(y^+) + B. \tag{10}$$

Resolving the wall means placing the first cell centre inside the viscous sublayer, typically targeting $y^+ \approx 1$, and using boundary conditions that integrate the turbulence equations to the wall. This requires many thin, high-aspect-ratio cells and several cells across the boundary layer. Wall functions avoid resolving the viscous sublayer. They impose an algebraic relation between the wall shear stress and the velocity at the first cell centre, reducing the near-wall cell count but requiring the first cell to lie in the range for which the wall function was developed. A mesh with an unsuitable y^+ can therefore produce misleading skin friction, separation and aerodynamic forces even when the outer flow appears converged.

The generated cases write turbulence wall boundary conditions from the repository's fluid wall-options template. The choice is made independently for each geometry and ground section with `WALL_MODEL` or `GRND_WALL_MODEL`. The available inputs are:

- `high`: selects `highReynolds`. The turbulent viscosity uses `nutkWallFunction`; k, ε and ω use their corresponding OpenFOAM wall functions. This is the conventional high-Reynolds-number wall-function treatment and should be paired with a wall-function mesh rather than a nominally wall-resolved mesh.
- `ally`: selects `allReynolds`. The turbulent viscosity uses `nutUSpaldingWallFunction`, with wall functions for k, ε and ω. The Spalding formulation provides a blended velocity relation across the near-wall range and is intended to be less dependent on a sharp switch between viscous and logarithmic laws. It does not remove the need to check y^+ or guarantee wall-resolved accuracy.
- `low`: selects `lowReynolds`. The turbulent viscosity uses `nutLowReWallFunction`; k, ε, ω and the Spalart–Allmaras variable use near-wall fixed values. This is the option intended for a low-y^+, wall-resolved treatment. It only behaves as such when the mesh actually resolves the viscous sublayer; selecting `low` on a coarse wall-function mesh does not create wall resolution.

For the Spalart–Allmaras model, the repository sets $\widetilde{\nu}$ to zero at the wall and applies the selected treatment to ν_t. For k–ω SST and GEKO, the k and ω fields use the corresponding OpenFOAM wall functions in the high- and all-Reynolds options. The k–ε template follows the same high-level wall-function approach, although that model is not currently exposed by the case selector.

There is no separate wall-resolved LES model or automatic wall-model selection in the current setup. The transient IDDES choices inherit wall treatment from their underlying Spalart–Allmaras or SST formulation; they are hybrid RANS–LES models, not wall-resolved LES by default. A `slip` ground patch has no no-slip wall shear and therefore does not use a turbulence wall function on that patch. Likewise, the `laminar` template has no turbulent stress model, so the turbulence wall-function discussion does not apply to a truly laminar case.

The solver reports the computed `yPlus` field through the standard function object. Always inspect its distribution on the vehicle, wheels and ground before interpreting wall shear or force coefficients. A single global average can hide regions of separation, stagnation or excessive cell size, so local maxima and the values over force-producing surfaces are important. Confirm the generated turbulence-properties file before production runs.

3.4 Other turbulence models in the template library

The repository also contains templates for hybrid RANS–LES and laminar calculations. These models do not have the same interpretation as a steady RANS mean solution.

- **LES**: Large Eddy Simulation resolves the large, energy-containing eddies and models the smaller, more universal scales with a subgrid-scale model. It requires substantially finer spatial and temporal resolution than RANS, particularly near walls.
- **DES**: Detached Eddy Simulation uses a RANS model near attached walls and switches toward LES behaviour in sufficiently resolved separated regions. It is intended to resolve large detached structures without the cost of wall-resolved LES everywhere.
- **DDES**: Delayed Detached Eddy Simulation adds shielding intended to prevent premature switching from RANS to LES inside attached boundary layers. The template library includes `SpalartAllmarasDDES`.
- **IDDES**: Improved Delayed Detached Eddy Simulation extends the hybrid approach with improved shielding and length-scale treatments. The transient aliases `sa` and `kosst` select `SpalartAllmarasIDDES` and `kOmegaSSTIDDES`, respectively.
- **Laminar**: A laminar calculation sets the turbulent contribution to zero and solves the molecular-viscosity equations only. The `laminar` template is present for template development, but it is not currently exposed by the `TURB_MODEL` selector.

The currently exposed choices are therefore `sa`, `kosst` and `geko` for steady cases, and `sa` and `kosst` for transient cases. The transient mapping uses hybrid models and should not be interpreted as a time-accurate RANS calculation. Confirm the generated `constant/turbulencePropertiesSolve` file and the installed ESI OpenFOAM version before production runs, especially when using hybrid models or custom templates.

4 Practical Guide

This chapter collects the installation, execution, verification and troubleshooting information needed for a reproducible caseSetup workflow. The examples use the repository layout shown in the introduction. Commands should be run from the caseSetup repository unless stated otherwise.

4.1 Installation and environment setup

caseSetup requires Python 3 and the packages listed in `requirements.txt`. The generated case must also be run with a compatible ESI OpenFOAM installation. The repository currently targets the ESI releases `of2206` and `of2606`. ParaView post-processing requires `pvbatch` or `pvpython`; the regular system Python interpreter cannot import `paraview.simple`.

4.1.1 Linux

Create an isolated Python environment and install the project requirements:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Load the ESI OpenFOAM environment in each shell used for meshing or solving. The exact command depends on the local installation. Check the installation before creating a case:

```
which blockMesh
which snappyHexMesh
which pvbatch
```

4.1.2 Automated OpenFOAM and ParaView installation

The repository provides `installOpenFOAM.sh` to automate a full local toolchain. The script downloads and compiles one or more ESI OpenFOAM versions from source in both double (DP) and single (SP) precision, downloads a matching ParaView binary release, and rewrites every `clusterDict` under `setupTemplates/` so that the `zeroTemplates`, `postUtilities` and `pvbatch` paths point at this repository and the freshly installed toolchain. Each requested OpenFOAM version is verified to exist on the ESI download server before anything is downloaded, so a mistyped version fails before continuing.

Compilation of ESI OpenFOAM is only supported on Linux, either native or through WSL. The ParaView download and the `clusterDict` rewrite work on any POSIX shell, so the script can also be used on other systems purely to wire up paths.

The most common usage builds a single version and installs the build prerequisites on an Ubuntu or Debian system:

```
./installOpenFOAM.sh --versions "v2406" --install-deps
```

To build several versions into a chosen install root using a large number of parallel jobs:

```
./installOpenFOAM.sh --versions "v2306 v2606" --prefix $HOME/openFoam --jobs 16
```

The precisions built and the integer label size can be selected explicitly. The following builds double precision only with 64-bit labels, which is useful for very large meshes:

```
./installOpenFOAM.sh --versions "v2406" --precision "DP" --label-size 64
```

A specific ParaView release can be requested when the default does not match the local cluster. The version, series directory and binary flavour are all configurable:

```
./installOpenFOAM.sh --versions "v2606" \
    --paraview-version 5.12.0 \
    --paraview-series v5.12 \
    --paraview-flavor osmesa-MPI-Linux-Python3.10-x86_64
```

When ParaView is already available system-wide, or a headless machine only needs the solver, the ParaView stage can be skipped while still building OpenFOAM and rewriting the dictionaries:

```
./installOpenFOAM.sh --versions "v2406" --skip-paraview
```

If OpenFOAM and ParaView are already installed and only the `clusterDict` paths need to be repointed at the current repository checkout, both build stages can be skipped. This is the fastest way to fix the paths after moving or cloning the repository:

```
./installOpenFOAM.sh --skip-openfoam --skip-paraview
```

The `clusterDict` rewrite normally also injects a `source` of the appropriate OpenFOAM `etc/bashrc` into the generated commands, matching each template to a build by the version tag in its directory name where present. This behaviour can be disabled when the environment is loaded by other means, such as a cluster module system:

```
./installOpenFOAM.sh --versions "v2606" --clusterdict-precision DP --no-foam-source
```

Finally, the effect of any invocation can be previewed without downloading, building or modifying files using `-dry-run`, which prints the actions and the paths that would be written:

```
./installOpenFOAM.sh --versions "v2306 v2406" --dry-run
```

A full list of options, including all defaults, is available through `./installOpenFOAM.sh -help`. As with any generated configuration, the rewritten `clusterDict` files and the built solver should be validated with a small sample case before production use.

4.1.3 Windows through WSL

Native Windows execution is not tested. The recommended Windows approach is Windows Subsystem for Linux (WSL), preferably with an Ubuntu distribution. Install WSL and the Linux distribution using Microsoft's current instructions, then perform the Linux setup inside the WSL terminal. Install Python, the required build tools, ESI OpenFOAM and ParaView inside WSL rather than mixing Windows and Linux executables.

The repository can be accessed through the mounted Windows drives under `/mnt`, but case files and temporary solver data should preferably be stored inside the WSL Linux filesystem for better I/O performance. Windows and WSL paths are not interchangeable: use Linux paths in shell commands and in generated case configuration. The WSL workflow is untested and should be validated with a small sample case before production use. In particular, verify file permissions, symbolic links, line endings, OpenFOAM availability and ParaView batch support.

4.2 Recommended case workflow

A standard case proceeds through the following stages:

1. Prepare and inspect the geometry in the reference mesh directory.
2. Select a setup template and create the case directory.
3. Write or edit the `caseSetup` input, exposing only the modules that need to differ from the defaults.
4. Run `caseSetup.py` to link geometry and write the OpenFOAM dictionaries.
5. Inspect the generated patches, boundary conditions, reference values and turbulence settings.
6. Generate the mesh and check mesh quality before solving.
7. Run the initialization and solver stages, locally or through the selected cluster template.
8. Check residuals, force histories, mass balance and `yPlus`.
9. Average the converged solution and run `pvPost.py` through `pvbatch` when images or CSV data are required.
10. Run `postRun.py` for force summaries, uncertainty estimates and optional wing-pressure plots.

Do not treat a successfully completed command as proof that the case is physically or numerically valid. The generated dictionaries and solver logs must be inspected after each major stage.

4.3 Input and unit conventions

The main input file is divided into modules such as GLOBAL_SIM_CONTROL, BC_SETUP, geometry sections, ride-height sections, cornering sections and post-processing sections. The detailed module descriptions are provided in the basic setup and post-processing chapters. Values should be checked against the units shown beside each input. In particular:

- velocities are in m/s;
- lengths and geometry coordinates are in the case length unit, normally m;
- pressures and FAN pressure rises are in Pa;
- angles are in degrees where explicitly labelled; and
- reference area, length and centre must use the same coordinate system as the geometry.

The SAE coordinate convention used by the setup should be confirmed before assigning drag, lift, yaw, pitch and roll directions. A sign error in a reference vector can produce plausible-looking but incorrectly signed coefficients.

4.4 Potential Examples

4.4.1 Straight-line external aerodynamics

Begin with the default template and a minimal geometry list. Set the inlet speed, reference area and reference length, select the moving or stationary ground condition, and leave the remaining modules at their defaults for the first test. Generate the case, inspect the boundary-condition file and complete a coarse mesh before increasing refinement.

4.4.2 Ride-height study

Remove the default ride-height module from DEFAULT_MODULES, define the ride-height points and run the base case setup first. The tool creates child cases and applies the calculated translations, pitch and roll. Check the transformation logs and verify that each child has the intended geometry before submitting the study.

4.4.3 Cornering case

Enable the cornering module and provide the turn radius, yaw direction and wheel or ground settings required by the case. Confirm that the relative velocity fields and SRF settings are present in the generated case. Force directions and post-processing variables must be interpreted in the rotating-frame context.

4.4.4 FAN and POR radiator case

Use a separate planar FAN disk and radiator POR region. Confirm that the fan surface is a single approximately planar disk, that its normal points toward the target geometry, and that the generated pressure-jump boundary condition matches the installed ESI OpenFOAM version. Dedicated FAN reporting is not currently implemented.

4.5 Verification and validation

Verification checks whether the numerical setup is solving the intended equations. Validation checks whether the result agrees with measurements or a trusted reference. At minimum, record the following for every case:

- mesh cell count, quality statistics and refinement settings;
- solver, OpenFOAM release, template name and caseSetup revision;

- turbulence model, wall model and local `yPlus` distribution;
- residual histories and force or moment histories;
- time-step or pseudo-time settings for transient cases;
- averaging start and end times or iterations; and
- reference area, length, centre, velocity and coefficient directions.

Residual reduction alone is not a convergence criterion for external aerodynamics. Inspect the stability of drag, lift, moments and relevant component forces over the averaging interval. Also check continuity or mass imbalance and make sure the averaging window excludes initialization and startup transients.

Use at least one mesh-sensitivity study and, for transient or hybrid calculations, a time-step or sampling-sensitivity study. Compare integrated forces, pressure distributions and important flow features rather than only residual values. For experimental comparison, document blockage, ground and wheel conditions, Reynolds number, reference quantities and uncertainty; otherwise an apparently large CFD error may be a difference in test definition.

4.6 Interpreting results

The coefficient summary and post-processing outputs should be interpreted together. Integrated coefficients can conceal local errors, so compare them with surface pressure, wall shear, wake structure and component-level forces. The pressure coefficient is conventionally defined as

$$C_p = \frac{p - p_\infty}{\frac{1}{2}\rho U_\infty^2}. \tag{11}$$

Use the wing-pressure extraction to compare C_p against streamwise position at each selected spanwise plane. Confirm that the extracted CSV files correspond to the intended geometry and that missing sections have been reported rather than silently treated as zero pressure. For wall shear and force interpretation, inspect the local `yPlus` field and the chosen wall treatment.

4.7 Cluster and automation checklist

Before submitting a cluster job:

1. Confirm that the selected cluster template loads the intended OpenFOAM release.
2. Check the requested core count against the decomposition settings.
3. Verify that geometry links and custom setup files are visible on compute nodes.
4. Confirm that the solver command, restart state and output directory are correct.
5. Enable the required sections in the case copy of `pvPostSetup` before post-processing.
6. Review the solver log and post-processing log after the job finishes.

A failed job should be restarted only after identifying whether the cause was a numerical instability, an invalid dictionary, a missing file, a resource limit or an environment problem. Preserve the original log when making a restart.

4.8 Customising the clusterDict

The execution scripts are not hand-written. For each case, `caseSetup.py` reads the `clusterDict` of the selected template from `setupTemplates/<template>/defaultCluster/slurm/clusterDict` and assembles the `meshingScript`, `solveScript`, `exportScript` and `postScript` files in the case directory. Editing the `clusterDict` is therefore the supported way to build a customised cluster or local workflow. The automated installer described earlier only rewrites specific paths inside these files; the structure below is what a user edits by hand.

4.8.1 Structure

A `clusterDict` is a single JSON object. Its top-level keys are the workflow stages, and each stage is itself an object whose values are shell-command fragments:

```
{
  "snappyHexMesh": {
     "preamble":   "...clean logs, set CORES...",
     "blockMesh":  "...; blockMesh >> log.blockMesh;",
     "snappyHexMesh": "...; mpirun -np $CORES snappyHexMesh -parallel -overwrite >> log.snappyHexMesh;"
  },
  "solve":  { "preamble": "...", "solve": { "steady": "...", "transient": "..." } },
  "export": { "export": "...", "exportEnsight": "..." },
  "post":   { "forces": "...", "postPro": "..." }
}
```

The stage keys map to the generated scripts as follows:

- `snappyHexMesh` or `ansaMesh` builds `meshingScript`. The mesher is chosen from the case `TEMPLATE_TYPE`: an `ansa` template uses the `ansaMesh` block, otherwise the `snappyHexMesh` block is used.
- `solve` builds `solveScript`.
- `export` builds `exportScript`.
- `post` builds `postScript`.

Within a stage, the command fragments are concatenated in the order they appear, separated by newlines, to form the script. The key names are labels for readability and ordering; only their order and contents matter. Because the fragments are joined verbatim, each fragment normally ends with a semicolon and typically begins with an error guard such as `if [ $? != 0 ]; then exit 1; fi;` so that a failure in one step stops the script.

4.8.2 Conditional and nested keys

A few keys inside the `solve` stage are selected by the case configuration rather than always emitted, and they use a nested object:

- `solve.initialize` contains `initializePotential` and `initializeSteady`; the entry matching `SIM_INIT` is inserted, and initialization is skipped entirely for cornering cases.
- `solve.solve` contains `steady` and `transient`; the entry matching `SIM_TYPE` is inserted.
- `solve.topoSet` is inserted only when the case defines porous or MRF cell zones.
- `solve.groundPatch` is inserted only when the case defines ground belt or plate patches.

Any other key in a stage is always included. Keep these reserved names and their nested structure when editing, otherwise the corresponding step will be omitted or a lookup will fail.

4.8.3 Environment and precision

The commands run in whatever environment the script establishes. The simple templates assume OpenFOAM is already sourced and refer to `$WM_PROJECT_DIR`; the installer can inject a `source` of the correct `etc/bashrc` into each stage. The OTR templates instead carry an explicit `precision` command at the start of each stage that sources OpenFOAM with a chosen precision. In both cases the meshing stage must run in double precision for `snappyHexMesh` robustness, while the solve stage may run in double or single precision. When writing a custom stage, source OpenFOAM the same way as the neighbouring commands in that template and keep meshing in double precision.

4.8.4 Adding a custom command

To add a step, insert a new key into the relevant stage. For example, to run `checkMesh` again and write a quality report immediately after meshing, add a key to the `snappyHexMesh` block after the existing `checkMesh` entry:

```
"customReport":"if [ $? != 0 ]; then exit 1; fi; echo 'writing mesh report...'; mpirun -np $CORES checkMesh -
    parallel -writeAllFields >> log.checkMeshReport;"
```

To call an external script of your own after the solver finishes, add a key to the `solve` or `post` block. Reference the repository through the same path style used by the neighbouring commands so that the installer and the compute nodes resolve it consistently:

```
"customPost":"echo 'running custom analysis...'; python3 /path/to/myAnalysis.py >> log.customPost;"
```

4.8.5 Building a new template

For a substantially different workflow, copy an existing template directory rather than editing a shared one:

```
cp -r setupTemplates/default setupTemplates/myWorkflow
```

Edit `setupTemplates/myWorkflow/defaultCluster/slurm/clusterDict` and select the template when creating the case. After editing, validate the JSON and inspect the generated scripts before submitting a job:

```
python3 -m json.tool setupTemplates/myWorkflow/defaultCluster/slurm/clusterDict > /dev/null
```

The most common editing mistake is invalid JSON, usually a missing comma between keys, an unescaped double quote inside a command, or a trailing comma after the last key. Validate the file, generate a case, and read the resulting `meshingScript`, `solveScript`, `exportScript` and `postScript` to confirm the commands and their order before running anything on a cluster.

4.9 Reproducibility and provenance

A case should be reproducible without relying on an individual's shell history. Store the input file, generated `fullCaseSetupDict`, selected template, mesh statistics, solver logs and post-processing configuration with the case record. Also record the caseSetup Git revision, OpenFOAM release, ParaView version, Python environment and any local template modifications.

Do not overwrite repository defaults to customize one case. Copy the relevant template or post-processing file into a case-specific location and record the change. For a completed study, retain the exact geometry archive, ride-height table, cluster submission settings and post-processing command-line arguments.

4.10 Extending the framework

The framework can be extended with new setup modules, geometry prefixes, solver templates, function objects, ParaView variables and post-processing commands. New extensions should follow the existing template-driven pattern:

1. define the input syntax and defaults;
2. add validation and a clear error message for invalid values;
3. write the generated dictionary without changing unrelated modules;
4. add a minimal example and document units and limitations; and
5. test both the generated text and a complete case using the target OpenFOAM release.

A new turbulence or wall model must be mapped in the solver-selection dictionaries and supplied with compatible field templates, initial fields, boundary conditions and solver settings. A template file by itself does not make a model selectable. Custom models should be marked as version-dependent and validated with the installed ESI release.

4.11 Troubleshooting

- **Missing geometry:** check the reference directory, file extension, compression and symbolic-link targets. Confirm that compute nodes can resolve the same paths.
- **Invalid module or option:** compare the spelling and case of the input key with the documented module. The error message lists valid choices for many selectors.
- **Mesh failure:** inspect surface closure, normals, scale, refinement transitions, layer settings and mesh-quality output. Reduce the problem to one geometry or a coarse refinement when isolating the cause.
- **Unexpected forces:** verify reference vectors, moving-ground and wheel motion, patch names, pressure reference and force-coefficient surfaces.
- **Poor wall resolution:** inspect local `yPlus` rather than only a global average, then change the first-cell height or wall model consistently.
- **Missing ParaView data:** run `pvPost.py` with `pvbatch`, verify that the requested fields were exported, and check that the selected variable exists in the custom dictionary.
- **FAN failure:** verify disk planarity, normal direction, target geometry, pressure-rise units and the generated pressure-jump dictionary against the ESI release.
- **WSL issue:** check Linux permissions, line endings, symbolic links, mounted-drive performance and that all commands resolve to WSL installations rather than Windows executables.

5 Basic Setup

To start, you would need a case directory created in your CASES directory. Once created, you will not have a caseSetup file just yet. To get a file, execute the following line in your case directory,

```
caseSetup --new
```

A fresh caseSetup file will be generated. The inside of the file will look like this:

```
[DEFAULTS]
DEFAULT_MODULES = defaultGlobalControl defaultGlobalSimControl defaultBCSetup defaultGlobalDecomposition
     defaultGlobalMaterial defaultGlobalRefinement defaultPost
ADDON_MODULES =

[TITLES]
CASENAME = casename
CASEDESCRP = default description
JOBCODE = TS

[GEOMETRY]
GEOM = default.stl,1,8,5,1.1,high
```

The DEFAULT_MODULES line controls the settings that are available. If the setting is not visible, it just means that it will use the default setup. Each setting here is explained below:

- CASENAME - the case name for your case, it does not do anything at this time, but in the future it could be added to the post processing images.
- CASEDESCRP - case description, see CASENAME
- JOBCODE - Two letter initial for the job you are running, just helps identify the job that's running on the cluster queue.
- GEOM - List of geometries to use in the case. A comma separates each setting for that particular geometry, the next geometry goes on the subsequent line.
  - What each column represents:

    ```
    GEOMETRY_FILE_NAME,SCALE,MESH_LEVEL,INFLATION_LAYERS,INFLATION_LAYER_GROWTH_RATE,
        WALL_MODEL
    ```

5.1 Creating a Custom Setup Template

The setup selected by caseSetup -setup is a directory under setupTemplates. To make a custom setup based on an existing setup, copy the complete source directory, including its defaultSetup, defaultBCTemplates, defaultDicts, defaultPost, defaultRefinements and defaultCluster directories.

For example, to create mySetup from the repository's default setup:

```
cd /path/to/caseSetup
cp -R setupTemplates/default setupTemplates/mySetup
```

Edit the copied files in setupTemplates/mySetup/defaultSetup/. The most commonly changed files are:

- defaultModules and defaultOrder - available modules and their order;
- defaultGlobalControl, defaultGlobalSimControl and defaultTitles - general case and run defaults;
- defaultGlobalRefinement and defaultGeometry - mesh and geometry defaults;
- defaultPVPost and defaultPost - post-processing defaults;
- defaultRideHeight and defaultCornering - optional ride-height and cornering defaults.

Select the copied setup by passing its directory name:

```
python caseSetup.py --setup mySetup
```

The value supplied to -setup must match the directory name exactly. For cluster use, update the copied defaultCluster/slurm/clusterDict with the machine-specific paths and commands. Changes to a template apply to cases generated or regenerated afterward; they do not modify existing generated cases automatically.

Each geometry that is required in this run and is in its own file should be listed here, the settings shown above will be applied to all geometries in that file. If a different settings are required to be applied, that geometry should be broken out into it's own file. A common example is wheels.

5.2 Rotating Geometries

The wheels and tires are spinning and therefore should have a rotating wall boundary condition applied, therefore it MUST be in its own part. Example,

```
[DEFAULTS]
DEFAULT_MODULES = defaultGlobalControl defaultGlobalSimControl defaultBCSetup defaultGlobalDecomposition
     defaultGlobalMaterial defaultGlobalRefinement defaultPost
ADDON_MODULES =

[TITLES]
CASENAME = casename
CASEDESCRP = default description
JOBCODE = TS

[GEOMETRY]
GEOM = body.obj.gz,1,8,5,1.1,high
        ROTA-fr-wh-lhs,1,8,5,1.1,high
        ROTA-fr-wh-rhs,1,8,5,1.1,high
        ROTA-rr-wh-lhs,1,8,5,1.1,high
        ROTA-rr-wh-rhs,1,8,5,1.1,high
```

Notice the prefix ROTA, it lets caseSetup know that this geometry will have a rotating wall boundary condition applied. Upon running caseSetup, if it's the first time with this set of geometry, caseSetup will stop and let you know that the following blocks have been added. The user should check that these are the appropriate settings for each wheel geometry.

NOTE: If using a plinth, ensure that the plinth is a separate geometry, else the automatic wheel center calculation will include the plinth in the radius calculation resulting in an incorrect radius and angular velocity!

```
[ROTA-fr-wh-lhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True

[ROTA-fr-wh-rhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True

[ROTA-rr-wh-lhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True

[ROTA-rr-wh-rhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True
```

- WH_RAD [[float,m]] - wheel radius, distance from wheel center to ground (r, default)
- WH_CENTER [[float,m]] [[float,m]] [[float,m]] - wheel center coordinates in global coordinates system (x y z, default)
- WH_AXIS [[float,m]] [[float,m]] [[float,m]] - wheel axis of rotation from, using right hand rule with respect to the global coordinate system (x y z, default)
- WH_AXIS [bool] - activate rotating wall or use no-slip condition (true, false)

Using default option will trigger the automatic calculation but manual input of values is still possible. To speed up caseSetup it is recommended to copy the values output by the calculation

in the termial and input them into the caseSetup if coordinates and axis values are not expected to change for subsequent runs.

5.3 Controlling Run Settings

Deleting the module `defaultGlobalControl` will expose the following block:

```
[GLOBAL_CONTROL]
STARTFROM = latestTime
STARTTIME = 0
ENDTIME = 1000
DT = 1
WRITEINT = 500
PURGEINT = 1
AVGSTART = 500
```

- `STARTFROM` [`str`] - which time to start the simulation at, `latestTime` will start the simulation at the lastest available time if running for an existing solution, `startTime` will have the simulation start at the value specified in `STARTTIME` (`latestTime`, `startTime`)
- `STARTTIME` [`float/int,s`] - time to start the simulation at, will be used if `STARTFROM` is specified as `startTime`, must use `int` if steady run is used, or `float` if a transient run is selected
- `ENDTIME` [`float/int,s`] - final time or time step to end the simulation at, must use `int` if steady run is used, or `float` if a transient run is selected
- `DT` [`float/int,s`] - timestep size, must use `1` if steady run is used, or any `float` if a transient run is selected
- `WRITEINT` [`float/int,s`] - interval which the data is written to disk for restart or post-processing use, must use `int` if steady run is used, or any `float` if a transient run is selected (**CAUTION:** saving too frequently can cause increased solve time and increased storage usage!)
- `PURGEINT` [`int`] - number of written timesteps to keep, will only keep `PURGEINT` number of time steps (aside from `STARTTIME` or `0`)
- `DT` [`float/int,s`] - when to start averaging flow variables, must use `int` if steady run is used, or `float` if a transient run is selected

5.4 Controlling Simulation Settings

Deleting the module `defaultGlobalSimControl` will expose the following block:

```
[GLOBAL_SIM_CONTROL]
SIM_TYPE = steady
SIM_INIT = potential
SIM_SYM = half
TURB_MODEL = kosst
INIT_P = default
INIT_K = default
INIT_OMEGA = default
INIT_NUT = default
INIT_NUTILDA = default
```

- `SIM_TYPE` [`str`] - what type of simulation (`steady`, `transient`)
- `SIM_INIT` [`str`] - what type of initialization, `steady` only available when `SIM_TYPE` = `transient` (`none`, `potential`, `steady`)
- `SIM_SYM` [`str`] - simulation symmetry type, reccomended to run in `half` if not doing yaw and vehicle is mostly symmetrical, use `full` if running internal flow (`half`, `full`)
- `TURB_MODEL` [`str`] - turbulence model applied, `transient` runs can only use `sa` ([`kosst`,`sa`])
  - `steady`
    - `kosst` - $k-\omega$ Shear Stress Transport ($k-\omega$ SST)

        * sa - Spalart-Allmaras
    - transient
        * sa - Spalart-Allmaras Delayed Detached Eddy Simulation (SADDES)

- INIT_P [float,Pa] - initial conditions for pressure, reccomended to leave in default unless specific values are required (float, default)
- INIT_K [float,m2s-2] - initial conditions for turublent kinetic energy, reccomended to leave in default unless specific values are required (float, default)
- INIT_OMEGA [float,s-1] - initial conditions for ω term, reccomended to leave in default unless specific values are required (float, default)
- INIT_NUT [float,m2s-1] - initial conditions for ν_t term, reccomended to leave in default unless specific values are required (float, default)
- INIT_NUTILDA [float,m2s-1] - initial conditions for $\tilde{\nu}$ term, reccomended to leave in default unless specific values are required (float, default)

5.5 Boundary Conditions

Deleting the module defaultBCSetup will expose the following block:

```
[BC_SETUP]
INLET_MAG = 15
YAW = 0
DRAG_VEC = default
LIFT_VEC = default
PITCH_VEC = default
GROUND = moving
REFCOR = 0.775 0 0
REFLEN = 1.55
REFWIDTH = 1
REFAREA = 1
POROSITY = yes
BASE_CELL_SIZE = 1.6
DOMAIN_SIZE = 48 16 16
DOMAIN_OFFSET = 0.5
FRT_WH_CTR = 0 0 0
```

- INLET_MAG [float,ms-1] - inlet velocity magnitude ([float])
- YAW [float,deg] - yaw angle, will only yaw in the positive direction, if SIM_SYM is half, this value not used (float)
- DRAG_VEC [[float,m] [float,m] [float,m]] - drag force vector, following SAE J1594 convention by default (x y z, default)
- LIFT_VEC [[float,m] [float,m] [float,m]] - lift force vector, following SAE J1594 convention by default (x y z, default)
- PITCH_VEC [[float,m] [float,m] [float,m]] - pitch moment vector, following SAE J1594 convention by default (x y z, default)
- GROUND [str] - ground plane boundary condition (moving,stationary)
- REFCOR [[float,m] [float,m] [float,m]] - point defining center of rotation, typically center of wheel projected to the ground (x y z)
- REFLEN [[float,m]] - reference length, typically wheelbase length (float)
- REFWIDTH [[float,m]] - reference width, typically trackwidth length (float)
- REFAREA [[float,m]] - reference area, typically frontal area or 1 to have calculations done in $C_D A$ (float)
- POROSITY [str] - to calculate forces including forces due to porous media, recommend to leave at yes (yes, no)

- BASE_CELL_SIZE [[float,m]] - base cell size of blockMesh, resulting mesh levels are calculated based off this number, recommend to leave at 1.6 (float)
- DOMAIN_SIZE [[float,m] [float,m] [float,m]] - dimensions for blockMesh domain (x y z)
- DOMAIN_OFFSET [[float,m]] - distance of FRT_WH_CTR to the inlet boundary as a ratio of domain x length, i.e. 0.5 means the FRT_WH_CTR is located at the center of the domain (float)
- FRT_WH_CTR [[float,m] [float,m] [float,m]] - coordinate for the center point projected to the ground of the front wheel axle (x y z)

5.6 Decomposition and Cluster Control

Deleting the module defaultGlobalDecomposition will expose the following block:

```
[GLOBAL_COMPUTE_SETUP]
QUEUE_TYPE = slurm
SCRIPT_VERSION = 1
NUM_NODES = 1
TASK_PER_NODE = 72
DECOMP = 18 2 2
```

- QUEUE_TYPE [str] - NOT ACTIVE, FOR FUTURE USE (str)
- SCRIPT_VERSION [int] - NOT ACTIVE, FOR FUTURE USE (int)
- NUM_NODES [int] - NOT ACTIVE, FOR FUTURE USE (int)
- TASK_PER_NODE [int] - number of cores to use (int)
- DECOMP [int int int] - decomposition pattern, dictates the number of divisions in each direction during decomposition, freestream direction should be the largest, the product of the 3 values must match TASK_PER_NODE (x y z)

5.7 Fluid Properties

Deleting the module defaultMaterial will expose the following block:

```
[GLOBAL_MATERIAL]
NPHASE = 1
MATERIAL_TYPES = air
VISCOSITY = 1.46e-5
```

- NPHASE [int] - NOT ACTIVE, FOR FUTURE USE (int)
- MATERIAL_TYPES [str] - NOT ACTIVE, FOR FUTURE USE (str)
- VISCOSITY [float, m2s-1] - fluid kinematic viscosity (float)

5.8 Meshing Control

Deleting the module defaultGlobalRefinement will expose the following block:

```
[GLOBAL_REFINEMENT]
TEMPLATE_TYPE = snappyTemplate1
MIN_GEOM_LEVEL = 8
REF_FEAT_ANGLE = 25
GEOM_REF_DIST = 0.01 0.05 0.1
GEOM_REF_LEVELS = 9 8 7
DEFAULT_WAKE_REF = defaultWake
LOC_IN_MESH = -1 -1 4
LAYER_TYPE = absolute
DEF_EX_RATIO = 1.2
REF_ANGLE = 25
FEAT_EDGE_ANGLE = 140
FEAT_EDGE_LEVEL_INC = 1
FIRST_LAYER_HEIGHT = 0.0001
FINAL_LAYER_HEIGHT = 0.001
FINAL_LAYER_RATIO = 0.3
```

- TEMPLATE_TYPE [str] - meshing template to use (str)

  - snappyHexMesh - template file must exist in setupTemplates/<caseSetup_template>/defaultDicts/
  - ansaMesh - TEMPLATE_TYPE string must include caseSetup template name as well as ansa separated by underscores i.e. otr_ansa_default. The ansa is used to trigger the use of ansaMesh. The template must be located in setupTemplates/<caseSetup_template>/defaultANSA/.
    The last value entry in TEMPLATE_TYPE is the name of the template within that path and can be of any length. Just NO SPACES!

- MIN_GEOM_LEVEL [int] - coarses mesh level to be used for all geometries in the case (int)
- REF_FEAT_ANGLE [float,deg] - angle where curvatures with angles larger than it would trigger mesh refinement in that area, operates only for ansaMesh (float)
- GEOM_REF_DIST [[float,m] [float,m] [float,m] ...] - distance from the surface which the mesh level from GEOM_REF_LEVELS shall be applied, unlimited entries, but must match length in GEOM_REF_LEVELS (float float ...)
- GEOM_REF_LEVELS [int int ...] - refinement levels for each corresponding distance in GEOM_REF_DIST unlimited entries, but must match length in GEOM_REF_DIST (int int ...)
- DEFAULT_WAKE_REF [str] - wake refinement generated by default or use specific template (true, false, str)
  - if using specific template, it should exist in setupTemplates/<caseSetup_template>/defaultRefinements/
- LOC_IN_MESH [[float,m] [float,m] [float,m]] - point which defines the "inside" location of the domain, only works for snappyHexMesh (float float)
- LAYER_TYPE [str] - how the layer growth occurs globally, only works for ansaMesh, defined in snappyHexMesh by template (absolute, aspect)
  - absolute - first layer height is defined absolutely based on FIRST_LAYER_HEIGHT, rest of inflation layer sizing determined by aspect ratio set for the geometry
  - aspect - first layer height is defined as a ratio to cell size outside of the inflation layers, global settings currently hardcoded as 0.5, rest of inflation layer sizing determined by aspect ratio set for the geometry
- DEF_EX_RATIO [float] - global layer expansion ratio (float)
- REF_ANGLE [float,deg] - angle where curvatures with angles larger than it would trigger mesh refinement in that area, operates only for snappyHexMesh (float)
- FEAT_EDGE_ANGLE [float,deg] - angle under which triggers edge refinement, only works for snappyHexMesh (float)
- FEAT_EDGE_LEVEL_INC [int] - refinement level increment for edges defined by FEAT_EDGE_ANGLE, only works for snappyHexMesh (int)
- FIRST_LAYER_HEIGHT [[float,m]] - first layer height, only works for ansaMesh (float)
- FINAL_LAYER_HEIGHT [[float,m]] - total height of inflation layers, not currently operational, for future use (float)
- FINAL_LAYER_RATIO [[float,m]] - ratio of last layer and the surrounding element size, not currently operational, for future use (float)

5.9 Post Processing Control

Deleting the module `defaultPost` will expose the following block:

```
[GLOBAL_POST_PRO]
ISOQ = 2000
ISOCPT = 0
SLICES = True
XSLICE = default
YSLICE = default
ZSLICE = default
```

- `ISOQ` [`float,s-2`] - Q criterion value of which to generate VTK iso surfaces of (`float`)
- `ISOCPT` [`float`] - $C_{p,total}$ value of which to generate VTK iso surfaces of (`float`)
- `SLICES` [`str`] - to generate VTK slices (`true`, `false`)
- `XSLICE` [[`float,m`] [`float,m`] [`float,m`]] - start, end and spacing of the x normal slices (`float`, `float`, `float`)
- `YSLICE` [[`float,m`] [`float,m`] [`float,m`]] - start, end and spacing of the y normal slices (`float`, `float`, `float`)
- `ZSLICE` [[`float,m`] [`float,m`] [`float,m`]] - start, end and spacing of the z normal slices (`float`, `float`, `float`)

5.10 Ride Height Control

Deleting the module `defaultRideHeight` will expose the following block that is under development currently. The goal is to create a process which can do ride height mapping.

```
[RIDE_HEIGHT_SETUP]
RUN_RIDE_HEIGHT = false
RIDE_HEIGHT_FILE =
USE_DEPEND = True
```

This section outlines the usage of the ride height mapping tool. A ride height map is useful to understand the dynamic sensitivity of an aero package as the car goes through the corners. We can use it to run in virtual lap time simulations as well. The use of the ride height mapping tool is diverse, we can use it for evaluation of a concept to understand the overall sensitivity from concept to concept. Some concepts perform better under braking, or under acceleration.

For the sake of simplicity, we make the assumption that there is enough time for the air flow to settle and there is no hysterisis from attitude to attitude on the track, although this might not be true. Therefore, each ride height is run statically with no movement in the simulation after transformation.

5.10.1 Ride Height Map Generation

Generating a ride height map is done by using on track or simulations data, looking at the front ride height vs rear ride height scatter plots and creating a point grid that encompases the data. There might be some outliers that rarely get reached, but it is helpful to add a point there to ensure the map is interpolated well. A full ride height map can be many points, which can result in very long total computational times. They should not be something run day to day. Rather a limit selection of ride heights can be run for conceptual analysis. The full map can be run at the end of a design cycle to evaluate the overall sensitivity.

For day to day usage, the ride height points can be limited to 3-5 points relevant to the geometry concept at hand. For example, a front wing change it might be more relevant to use select nose down and nose down plus roll conditions in addition to a level condition.

5.10.2 Ride Height Tool Operation

The ride height tool operates by reading a `.csv` file containing front left (`fl`), front right (`fr`), rear left (`rl`), and rear right (`rr`) ride heights as a deviation from the baseline condition. The ride height translation is done with a car centric coordinate system, meaning the car body does not move, only the wheels do as well as the domain. This gives the easiet comparison between ride heights in post processing. The ride height reference points are currently referenced to the axle location in `x` and `z` directions at the wheel base center in `y`. This is also assuming your front and rear wheelbases are the same. Future iterations will include alternate reference point locations.

Additionally, the model used for the ride height mapping should not contain suspension components as they will not be transformed at this time. Future iterations might include suspension kinematic transformations.

5.10.3 Ride Height Tool Usage

The use the ride height tool, delete the `defaultRideHeight` module in the `DEFAULT_MODULES` line in `caseSetup`. This will reveal the ride height module in `caseSetup`.

6 Prefix Controls

There are 3 or 4 letter prefixs that allow special control of your geometry and simulation. These help with assigning porous media regions, Multiple Reference Frame (MRF) regions, or even more control over meshing for certain geometries.

6.1 Rotating Geometries

The ROTA prefix allows the assignment of rotating wall to geometries such as wheels. A usage example is shown below.

```
[DEFAULTS]
DEFAULT_MODULES = defaultGlobalControl defaultGlobalSimControl defaultBCSetup defaultGlobalDecomposition
     defaultGlobalMaterial defaultGlobalRefinement defaultPost
ADDON_MODULES =

[TITLES]
CASENAME = casename
CASEDESCRP = default description
JOBCODE = TS

[GEOMETRY]
GEOM = body.obj.gz,1,8,5,1.1,high
        ROTA-fr-wh-lhs,1,8,5,1.1,high
        ROTA-fr-wh-rhs,1,8,5,1.1,high
        ROTA-rr-wh-lhs,1,8,5,1.1,high
        ROTA-rr-wh-rhs,1,8,5,1.1,high
```

Notice the prefix ROTA, it lets caseSetup know that this geometry will have a rotating wall boundary condition applied. Upon running caseSetup, if it's the first time with this set of geometry, caseSetup will stop and let you know that the following blocks have been added. The user should check that these are the appropriate settings for each wheel geometry.

NOTE: If using a plinth, ensure that the plinth is a separate geometry, else the automatic wheel center calculation will include the plinth in the radius calculation resulting in an incorrect radius and angular velocity!

```
[ROTA-fr-wh-lhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True

[ROTA-fr-wh-rhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True

[ROTA-rr-wh-lhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True

[ROTA-rr-wh-rhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True
```

- WH_RAD [[float,m]] - wheel radius, distance from wheel center to ground (r, default)
- WH_CENTER [[float,m]] [[float,m]] [[float,m]] - wheel center coordinates in global coordinates system (x y z, default)
- WH_AXIS [[float,m]] [[float,m]] [[float,m]] - wheel axis of rotation from, using right hand rule with respect to the global coordinate system (x y z, default)
- WH_AXIS [bool] - activate rotating wall or use no-slip condition (true, false)

6.2 Fan Pressure-Jump Condition

The prefix FAN defines a geometry-driven circular fan disk in a snappyHexMesh case. The disk is meshed as a baffle/face zone and is assigned the ESI OpenFOAM fan pressure-jump condition.

This is intended for applications such as an electric fan mounted upstream of a radiator. It is independent of the POR radiator model: a radiator may be represented by a porous volume, while the fan is represented by a separate planar disk.

The FAN geometry must be a single, approximately planar circular surface in an ASCII OBJ or STL file. It should not contain the fan frame, motor, blades, radiator, or other solids. The geometry file is supplied in the normal [GEOMETRY] section, using the FAN- prefix. FAN geometry is currently handled by the snappyHexMesh path; ANSA fan meshing is not enabled.

For example, the following setup places a fan disk upstream of a porous radiator. With default values, the center and equivalent circular diameter are calculated from the disk, and the disk normal is oriented toward the target radiator geometry.

```
[GEOMETRY]
GEOM = body.obj.gz,1,8,5,1.1,high
       POR-radiator.obj.gz,1,10,6,1.4,high
       FAN-radiator-fan.obj.gz,1,10,0,1.0,high

[FAN-radiator-fan]
FAN_MODEL = fan
FAN_CENTER = default
FAN_DIRECTION = default
FAN_TARGET_GEOMETRY = POR-radiator
FAN_DIAMETER = default
FAN_PRESSURE_RISE = 250
SAMPLE_FAN = True
```

The FAN configuration block is exposed automatically the first time the geometry is encountered. The available settings are:

- FAN_MODEL [str] - ESI fan model selector. Use fan (pressureJump is accepted as an alias).
- FAN_CENTER [float,m] - disk center in global coordinates, or default to use the area-weighted surface centroid.
- FAN_DIRECTION [float] - three-component direction vector, or default to orient the disk normal toward FAN_TARGET_GEOMETRY. Explicit direction takes precedence over the target geometry.
- FAN_TARGET_GEOMETRY [str] - geometry name without its extension. Required when FAN_DIRECTION = default; for example, POR-radiator.
- FAN_DIAMETER [float,m] - disk diameter, or default to calculate the equivalent circular diameter from surface area.
- FAN_PRESSURE_RISE [float,Pa] - constant pressure jump applied by the ESI fan condition. The example applies a 250 Pa pressure rise. Pressure curves are not currently accepted by the caseSetup configuration; only one scalar value is supported.
- SAMPLE_FAN [bool] - fan sampling switch retained in the setup metadata; FAN reporting is not yet implemented.

The generated pressure boundary uses the ESI syntax type fan, patchType cyclic, and a constant jumpTable. The pressure-rise sign follows the resolved FAN direction. If the fan accelerates flow in the opposite direction, provide an explicit FAN_DIRECTION or reverse the target geometry arrangement before running the case. Verify the generated boundary condition with the installed ESI OpenFOAM release (of2206 or of2606) before production use.

Using default option will trigger the automatic calculation but manual input of values is still possible. To speed up caseSetup it is recommended to copy the values output by the calculation in the termial and input them into the caseSetup if coordinates and axis values are not expected to change for subsequent runs.

6.3 Porous Media

When radiators are used in a simulation, it would be very computationally expensive to model each radiator fin. So instead we use the Darcy-Forcheimer porous media calculation to model the pressure

drop across a radiator which allows us to estimate the mass flow across the radiator and resulting forces.

The prefix POR is used to define the porous media geometry. **CAUTION:** The porous media geometry must only contain the "box" or region that represents the radiator fin region, do not include the frame or any other part of the radiator. The box which represents the fin region should be a simple prism. The inlet and outlets should have their own PID with the keyword -inlet or -outlet. There should only be 1 of each PID!

The mass flow calculations are performed on each pid in the porous media, therefore including the walls or any other surfaces where flow isn't moving *across* would make the calculations erronous.

Including the prefix POR on a geometry would expose the following block:

```
GEOM = trial005-rad.obj.gz,1,8,6,1.4,high
       fw-H05A00.obj.gz,1,8,6,1.4,high
       rw-R11I01.obj.gz,1,8,6,1.4,high
       POR-rad-001.obj.gz,1,10,6,1.4,high
       ROTA-fr-wh-lhs-v3-3.obj.gz,1,8,6,1.4,high
       ROTA-fr-wh-rhs-v3-3.obj.gz,1,8,6,1.4,high
       ROTA-rr-wh-lhs-v3-3.obj.gz,1,8,6,1.4,high
       ROTA-rr-wh-rhs-v3-3.obj.gz,1,8,6,1.4,high

[POR-rad-001]
DCOEFFS = 1000
FCOEFFS = 1000
VEC1 = 0.7071 0 -0.7071
VEC2 = 0 1 0
POINT = 1.7875 -0.000583 0.342739
SAMPLE_POR = True
```

the name of the block will be the name of the geometry just without the extension to identify it.

- DCOEFFS [float] - Darcy coefficient attained from experimental testing of pressure drop vs flow rate of the radiator (float)
- FCOEFFS [float] - Forcheimer coefficient attained from experimental testing of pressure drop vs flow rate of the radiator (float)
- VEC1 [[float,m] [float,m] [float,m]] - vector defining the normal of the radiator exit (float float float)
- VEC2 [[float,m] [float,m] [float,m]] - vector in plane with the radiator exit (float float float)
- POINT [[float,m] [float,m] [float,m]] - point defining the "inside" of the radiator (float float float)
- SAMPLE_POR [str,m] - to sample the mass flow through the radiator (true, false)

7 Ride-Height and Suspension Kinematics

caseSetup can automatically generate a family of child cases, each at a different ride-height state, and deform the geometry to follow the actual suspension motion. There are two ways this is done:

- simple - the wheels (and any ROTA geometry) are translated vertically by the requested corner displacement. No suspension linkage is required.
- kinematic - a double wishbone and pushrod kinematic solver computes the realistic motion of every suspension component (wheel camber and toe, control arm rotation, rocker angle, damper stroke) and applies a per-component transform to the matching parts of the geometry.

The mode is selected by the USE_KINEMATIC_SOLVER flag described below.

7.1 Ride-Height Setup

Deleting the module defaultRideHeight will expose the following block:

```
[RIDE_HEIGHT_SETUP]
RUN_RIDE_HEIGHT = False
RIDE_HEIGHT_FILE =
RUN_RH_POINTS = 1 2 3 4 5
USE_DEPEND = True
RH_UNIT = mm
INIT_RH =
RH_REF_WIDTH =
RH_REF_LEN =
USE_KINEMATIC_SOLVER = False
HARDPOINT_FILE = rideHeightUtils/suspensionHardpoints_template.cfg
HARDPOINT_SCALE = 0.001
SUSP_REQUIRE_ALL_PIDS_MATCHED = False
```

- RUN_RIDE_HEIGHT [bool] - master switch, when True, running caseSetup will generate one child case per requested ride-height point (True, False)
- RIDE_HEIGHT_FILE [str] - name of the ride-height file describing each point, see Section The Ride-Height File (str)
- RUN_RH_POINTS [int] - space separated list of point indices, matching the point column of the ride-height file, that will actually be built, allows running a subset of the table (int int ...)
- USE_DEPEND [bool] - reuse child case dependencies and links when they are already present (True, False)
- RH_UNIT [str] - unit of the corner displacements in the ride-height file (mm, m)
- INIT_RH [float] - optional initial ride-height offset applied to all points (float)
- RH_REF_WIDTH [[float,m]] - optional reference track width used when converting corner displacements into pitch, roll and heave (float)
- RH_REF_LEN [[float,m]] - optional reference wheelbase used when converting corner displacements into pitch, roll and heave (float)
- USE_KINEMATIC_SOLVER [bool] - enable the double wishbone and pushrod solver and the per-component geometry deformation, when False only simple vertical wheel translation is applied (True, False)
- HARDPOINT_FILE [str] - path to the suspension hardpoint definition file, required when USE_KINEMATIC_SOLVER is True (str)
- HARDPOINT_SCALE [float] - multiplier applied to every hardpoint coordinate to convert it to metres, use 0.001 if the hardpoint file is written in millimetres or 1.0 if it is already in metres (float)

- SUSP_REQUIRE_ALL_PIDS_MATCHED [bool] - when True, caseSetup stops with an error if any geometry part fails to match a suspension component, when False unmatched parts are left untouched and a warning is logged (True, False)

To run a ride-height study, set RUN_RIDE_HEIGHT to True and run caseSetup as normal. Child cases are then created and built automatically.

NOTE: -rideHeightMode is an internal flag used by caseSetup when it re-invokes itself inside each generated child case, it should not be supplied by the user.

7.2 The Ride-Height File

The ride-height file is a comma separated table where each row is one suspension state. The corner columns are the vertical wheel displacements in the unit set by RH_UNIT.

```
point,fl,fr,rl,rr,yaw,steer
1,0,0,0,0,0,0
2,-10,-10,0,0,0,5
3,-20,-20,0,0,0,0
4,0,-40,10,-30,0,0
5,-10,-10,-10,-10,0,0
```

- point [int] - unique index for the state, referenced by RUN_RH_POINTS and used to name the child case, for example 001_2 (int)
- fl, fr, rl, rr [[float]] - front-left, front-right, rear-left and rear-right wheel vertical displacements, negative values typically indicate compression (bump) and positive values rebound (droop) (float)
- yaw [[float,deg]] - vehicle yaw angle for that state, 0 for straight-line (float)
- steer [[float,deg]] - optional front road-wheel steering angle for that state, see Section Steering, 0 or omitted for no steer (float)

7.3 Steering

When the kinematic solver is enabled, an optional steer column in the ride-height file drives the front suspension through its steering rack. The value is the road-wheel angle, in degrees, requested at the reference wheel (the front-left). caseSetup solves for the rack travel that produces that angle at the front-left, then applies the same rack travel to the front-right. Because both front wheels share one rack but sit at different ride heights, the front-right angle is solved from the geometry rather than copied, so the natural Ackermann difference between the two wheels appears automatically. The rear wheels are not steered.

The steer is a true rack motion, not a rigid wheel rotation. The tie rod inner (rack) joint slides along the rack axis, the tie rod outer joint follows the upright, and the tie rod length is preserved. The wheel, upright and tie rod therefore move as a real linkage, and the resulting wheel pose feeds the same per-component transforms used for ride height.

The rack pushes the tie rod inner joint along the lateral (y) direction by default. This can be overridden per corner with the optional RACK_AXIS key in the hardpoint file. If the steer column is absent or all zero, the solver behaves exactly as a passive ride-height study.

NOTE: The steer value is the absolute road-wheel angle at the front-left, not an offset added to the static toe. A positive value steers the front-left wheel so its forward direction rotates toward +y, flip the sign in the ride-height file to steer the other way.

7.4 Suspension Hardpoint File

When the kinematic solver is enabled, the geometry of each corner's linkage is described in the hardpoint file pointed to by HARDPOINT_FILE. A template is provided at rideHeightUtils/suspensionHardpoints_templat
It uses the same format as the caseSetup file, with one section per corner, [SUSP_FL], [SUSP_FR], [SUSP_RL] and [SUSP_RR]. Each corner section looks like this:

```
[SUSP_FL]
WHEEL_CENTER_STATIC = 0 -781.03 356.57
UCA_F_INNER = -64.67 -157.20 500.45
UCA_R_INNER = 417.86 -157.20 488.42
LCA_F_INNER = -89.37 -157.20 330.51
LCA_R_INNER = 416.49 -157.20 350.42
TIE_INNER = -127.85 -157.20 330.51
UCA_OUTER_STATIC = 33.03 -707.27 520.88
LCA_OUTER_STATIC = 12.36 -737.95 331.40
TIE_OUTER_STATIC = -58.25 -706.55 461.63
PUSHROD_OUTER_STATIC = 12.36 -737.95 331.40
ROCKER_PIVOT = 61.44 -77.24 544.58
ROCKER_AXIS = 1.0 0.0 0.0
ROCKER_PUSHROD_JOINT_REF = 61.09 -159.93 558.55
ROCKER_DAMPER_JOINT_REF = 61.44 -77.24 614.58
ROCKER_DAMPER_CHASSIS = 61.44 0 544.58
WHEEL_AXIS_LOCAL = 0.0 1.0 0.0
WHEEL_FORWARD_LOCAL = 1.0 0.0 0.0
FL_UCA_PID = fluca
FL_LCA_PID = fllca
FL_ROCKER_PID = flrocker
FL_PUSHROD_PID = flprod
FL_DAMPER_PID = fldamper
FL_TIE_PID = fltie
FL_WHEEL_PID = flwheel
```

All coordinates are given in the global coordinate system, in the units implied by HARDPOINT_SCALE, for example millimetres when HARDPOINT_SCALE is 0.001.

- WHEEL_CENTER_STATIC [[float,m] [float,m] [float,m]] - static wheel center position (x y z)
- UCA_F_INNER, UCA_R_INNER [[float,m] [float,m] [float,m]] - upper control arm front and rear chassis (inner) mounts (x y z)
- UCA_OUTER_STATIC [[float,m] [float,m] [float,m]] - upper control arm outer (upright) ball joint (x y z)
- LCA_F_INNER, LCA_R_INNER [[float,m] [float,m] [float,m]] - lower control arm front and rear chassis (inner) mounts (x y z)
- LCA_OUTER_STATIC [[float,m] [float,m] [float,m]] - lower control arm outer (upright) ball joint (x y z)
- TIE_INNER, TIE_OUTER_STATIC [[float,m] [float,m] [float,m]] - tie rod inner (chassis or rack) and outer (upright) mounts (x y z)
- PUSHROD_OUTER_STATIC [[float,m] [float,m] [float,m]] - pushrod to upright (or pushrod to lower control arm) joint (x y z)
- ROCKER_PIVOT [[float,m] [float,m] [float,m]] - rocker rotation center (x y z)
- ROCKER_AXIS [[float,m] [float,m] [float,m]] - rocker rotation axis, using the right hand rule (x y z)
- ROCKER_PUSHROD_JOINT_REF, ROCKER_DAMPER_JOINT_REF [[float,m] [float,m] [float,m]] - the points on the rocker where the pushrod and damper attach (x y z)
- ROCKER_DAMPER_CHASSIS [[float,m] [float,m] [float,m]] - the chassis side damper mount (x y z)
- WHEEL_AXIS_LOCAL, WHEEL_FORWARD_LOCAL [[float,m] [float,m] [float,m]] - local spin axis and forward direction of the wheel, used to resolve camber and toe (x y z)
- RACK_AXIS [[float,m] [float,m] [float,m]] - optional steering rack travel direction, the tie rod inner joint slides along this axis when a steer angle is requested, defaults to lateral 0 1 0 if omitted (x y z)

7.5 Component PID Keywords

Any key in a corner section whose name contains PID is treated as a part name keyword. Its value is matched against the group or solid names inside the OBJ or STL geometry to decide which faces belong to which suspension component. The match is case insensitive and is satisfied when the keyword appears as a substring of the part name.

- FL_WHEEL_PID [str] - part keyword for the wheel (str)
- FL_UCA_PID [str] - part keyword for the upper control arm (str)
- FL_LCA_PID [str] - part keyword for the lower control arm (str)
- FL_ROCKER_PID [str] - part keyword for the rocker (str)
- FL_PUSHROD_PID [str] - part keyword for the pushrod (str)
- FL_DAMPER_PID [str] - part keyword for the damper (str)
- FL_TIE_PID [str] - part keyword for the tie rod (str)

The same set of keys exist for the other corners using the FR, RL and RR prefixes. The keyword is automatically classified into a component type. Classification accepts the full component name as well as common abbreviations, so values such as flprod or rlpr are correctly recognised as a pushrod. The recognised keywords for each component are:

- UCA - uca
- LCA - lca
- ROCKER - rocker, rock, rkr
- PUSHROD - pushrod, prod, pshrd, push, pr
- DAMPER - damper, damp, shock, strut
- WHEEL - wheel, whl
- TIE - tie, tierod, trod

NOTE: The keyword must be a substring of the actual part name in your geometry. For example a part group named body_susp_fllca is matched by the keyword fllca. If a part is not matched it is left in its static position. Set SUSP_REQUIRE_ALL_PIDS_MATCHED to True to turn unmatched parts into a hard error instead.

7.6 Optional Component Sections

Additional sections can attach extra geometry to a corner without that geometry being one of the primary kinematic parts. Any section that defines a CORNER key contributes its PID keywords to that corner. A usage example is shown below.

```
[SUSP_COMP_EXAMPLE_1]
CORNER = fl
PID = fluca_bracket
```

- CORNER [str] - which corner this geometry belongs to (fl, fr, rl, rr)
- PID [str] - part keyword for the extra geometry, any key whose name contains PID is accepted (str)

These sections do not introduce a transform of their own. They reuse the same machinery as the primary parts, so the choice of which component an optional part moves with is made in two steps:

- **Which corner** - given by the CORNER key. The section's PID keywords are merged into that corner's keyword pool, exactly as if they had been written directly inside the matching corner section, for example [SUSP_FL].

- **Which component** - given by the text of the keyword itself. The keyword is classified using the same recognised keywords listed under `Component PID Keywords`, and the optional part inherits that component's translation, rotation and pivot.

In the example above, `fluca_bracket` contains `uca`, so it is classified as an upper control arm and rides with the front-left upper control arm transform. In the same way a keyword such as `frdamper_shroud` contains `damper` and follows the front-right damper, including its length morph, and a keyword containing `wheel` follows the wheel center motion.

NOTE: If an optional keyword contains none of the recognised component keywords, for example `fl_aero_fairing`, it is not classified into any component, so no transform is assigned and the part is copied unchanged. To make an optional part follow a component, ensure its keyword contains the relevant keyword such as `uca`, `damper` or `wheel`.

7.7 Per-Component Transforms

For each ride-height point and corner the solver produces a transform for every component, made up of a translation, a rotation about an axis and a pivot point. How each component moves is summarised below.

- `WHEEL` - translates by the wheel center displacement and rotates with the camber and toe from the solver, about the static wheel center
- `UCA` - rotates as its outer joint follows the wheel, about the midpoint of its inner chassis mounts
- `LCA` - rotates as its outer joint follows the wheel, about the midpoint of its inner chassis mounts
- `ROCKER` - rotates in place by the rocker angle about the rocker axis, about the rocker pivot
- `PUSHROD` - inherits the motion of the rocker, about the rocker pivot
- `DAMPER` - inherits the motion of the rocker and stretches by the damper stroke, about the rocker pivot
- `TIE` - rotates as its outer end follows the wheel, about the tie rod inner mount, and when steering the inner end slides along the rack axis

For every transformed geometry a log is written next to the output file, `*.susp_pid_transform.json` and `*.susp_pid_transform.csv`, listing each part, the keyword it matched, and the applied translation and rotation. Inspect these to confirm the expected parts were moved.

7.8 Workflow Summary

- Set `RUN_RIDE_HEIGHT` to `True` and point `RIDE_HEIGHT_FILE` at your ride-height file.
- If kinematic motion is required, set `USE_KINEMATIC_SOLVER` to `True`, copy and edit the hardpoint template, and set `HARDPOINT_FILE` and `HARDPOINT_SCALE`.
- Ensure the geometry part names contain the PID keywords from the hardpoint file.
- Run `caseSetup` as normal. Child cases named `<case>_<point>` are created, the geometry is deformed, and `caseSetup` is re-run automatically inside each child.
- Each child case is then a standard, ready to mesh case at that suspension state.

7.9 Troubleshooting

- `PID scan skipped` - a geometry could not be processed by the PID aware path, for example an unsupported or binary format, so the file falls back to simple wheel translation. Check that the suspension geometry is ASCII OBJ or ASCII STL.
- `Unmatched PID` - the keyword from the hardpoint file does not appear in any part name. Confirm the keyword is a substring of the actual part name, or add a recognised keyword.

- `Geometry not moving as expected` - verify `HARDPOINT_SCALE` matches the unit of your hardpoint coordinates, and review the per-part transform log.
- `Kinematic solver skipped` - confirm `USE_KINEMATIC_SOLVER` is `True`, that `HARDPOINT_FILE` exists relative to the case, and that all required hardpoint keys are present.

8 Cornering (Single Reference Frame)

caseSetup can set up a steady curved-path (cornering) case using a Single Reference Frame (SRF). Instead of meshing a curved tunnel, the existing rectangular domain is solved in a frame that rotates at a constant angular velocity about a vertical axis through the corner centre. In that rotating frame the car follows a circular path of radius CORNER_RADIUS at the freestream speed INLET_MAG, and the steady flow field represents a mid-corner condition. The solver follows SIM_TYPE: a steady case uses SRFSimpleFoam and a transient case uses SRFPimpleFoam.

8.1 Cornering Setup

Deleting the module defaultCornering will expose the following block:

```
[CORNERING_SETUP]
RUN_CORNERING = False
CORNER_RADIUS =
CORNER_DIR = left
CORNER_CENTER = default
CORNER_CLEARANCE_FACTOR = 3
CORNER_SOLVER = SRFSimpleFoam
```

- RUN_CORNERING [bool] - master switch, when True the case is built as an SRF cornering case (True, False)
- CORNER_RADIUS [[float,m]] - radius of the corner the car is negotiating, measured from the corner centre to the reference centre of rotation, the frame angular velocity is INLET_MAG / CORNER_RADIUS (float)
- CORNER_DIR [str] - direction of the turn, left places the corner centre on the car's left (-y) and right on the car's right (+y) (left, right)
- CORNER_CENTER [str or [float,m] [float,m] [float,m]] - corner centre location, default derives it from the reference centre of rotation with a lateral offset of CORNER_RADIUS, or supply three floats to set it explicitly (default, or x y z)
- CORNER_CLEARANCE_FACTOR [float] - minimum ratio of CORNER_RADIUS to the domain lateral half-width, guards against a turn so tight that the rectangular box is a poor model of the curved tunnel (float)
- CORNER_SOLVER [str] - the OpenFOAM SRF solver used for the cornering run, the steady default is SRFSimpleFoam, when SIM_TYPE is transient it is auto-upgraded to SRFPimpleFoam (SRFSimpleFoam, SRFPimpleFoam)

To run a cornering case, set RUN_CORNERING to True, provide a valid CORNER_RADIUS and CORNER_DIR, and run caseSetup as normal.

NOTE: The cornering flow field is not laterally symmetric, so the case is always built full width. If the case is configured for half symmetry, caseSetup automatically promotes it to a full domain when RUN_CORNERING is True.

8.2 The Corner Centre

With CORNER_CENTER set to default, the corner centre is derived from the reference centre of rotation REFCOR (the wheelbase centre, at y = 0, projected to the ground). The centre is placed a lateral distance of CORNER_RADIUS to the side set by CORNER_DIR:

- CORNER_DIR = left - centre at REFCOR with y = REFCOR_y - CORNER_RADIUS
- CORNER_DIR = right - centre at REFCOR with y = REFCOR_y + CORNER_RADIUS

Deriving the centre from REFCOR makes the car pivot about its wheelbase centre rather than the front axle. The rotation axis is the tilted ground normal, the same normal used by the ride-height attitude, so the rotating frame stays aligned with the floor when the domain is pitched or rolled. Set CORNER_CENTER to three explicit floats to override the derived centre entirely.

8.3 Wheel Rotation in a Corner

In the rotating frame each wheel sits at a different distance from the corner axis, so the inner wheels travel a shorter path than the outer wheels. caseSetup sets each wheel's rotating-wall speed to the local ground speed at that wheel, equal to the frame angular velocity multiplied by the horizontal distance from the corner axis to the wheel centre. Inner wheels therefore spin slower and outer wheels faster than they would in a straight-line run at INLET_MAG.

8.4 Cornering Driven by the Ride-Height Map

Cornering can be combined with a ride-height study so that each ride-height point runs at the corner that physically matches its attitude. When both RUN_RIDE_HEIGHT and RUN_CORNERING are True, caseSetup looks for two optional columns in the ride-height file and applies them per point:

- corner_radius [[float,m]] - per-point corner radius, overrides CORNER_RADIUS for that child case, also accepted as cornerradius, corner_r or radius (float)
- corner_dir [str] - per-point turn direction, overrides CORNER_DIR for that child case, also accepted as cornerdir, turn_dir or direction (left, right)

An example ride-height file pairing each attitude with its corner is shown below.

```
point,fl,fr,rl,rr,yaw,steer,corner_radius,corner_dir
1,0,0,0,0,0,0,100,left
2,-10,-10,0,0,0,-5,100,left
3,-20,-20,0,0,0,-5,80,right
```

Each child case is generated with its own corner radius and direction, so it produces the matching corner centre, frame angular velocity and per-wheel rotation. If the corner_radius column is absent, every point falls back to the base CORNER_RADIUS, and likewise for corner_dir. A value that cannot be parsed, or a direction that is not left or right, is reported as a warning and falls back to the base setting for that point.

NOTE: A valid base CORNER_RADIUS is still required even when a corner_radius column is provided. The base value is validated before the child cases are generated and is used as the default for any point that does not supply its own radius.

8.5 Domain Validity

A rectangular box only approximates a curved tunnel when the turn radius is large relative to the box width. caseSetup enforces

```
CORNER_RADIUS >= CORNER_CLEARANCE_FACTOR * (DOMAIN_SIZE_y / 2)
```

and stops with an error reporting the minimum admissible radius if the check fails. The domain is never resized automatically. If a radius is rejected, increase CORNER_RADIUS, reduce the lateral domain size DOMAIN_SIZE y, or lower CORNER_CLEARANCE_FACTOR. When cornering is driven by the ride-height map, this check is applied to every point so an unsuitable per-point radius is caught early.

8.6 Workflow Summary

- Set RUN_CORNERING to True, and provide CORNER_RADIUS and CORNER_DIR.
- Leave CORNER_CENTER as default to pivot about the wheelbase centre, or set three floats to place the centre explicitly.
- To sweep attitudes and corners together, also set RUN_RIDE_HEIGHT to True and add corner_radius and corner_dir columns to the ride-height file.
- Run caseSetup as normal. The case is built full width, the SRF rotation is written, the wheels are set to their local corner speeds, and the run uses CORNER_SOLVER (SRFSimpleFoam for steady, SRFPimpleFoam for transient).

8.7 Troubleshooting

- `CORNER_RADIUS too small for this domain` - the clearance check failed. Increase `CORNER_RADIUS`, reduce `DOMAIN_SIZE y`, or lower `CORNER_CLEARANCE_FACTOR` to at least the reported minimum radius.
- `CORNER_RADIUS must be a positive number` - `RUN_CORNERING` is `True` but no valid base radius was given. Set `CORNER_RADIUS` directly, or provide a `corner_radius` column in the ride-height file together with a base value.
- `CORNER_DIR must be left or right` - the turn direction was not recognised. Use `left` or `right`, or correct the `corner_dir` column in the ride-height file.
- `Per-point corner value ignored` - a `corner_radius` or `corner_dir` entry could not be parsed and the base setting was used for that point. Check the column values in the ride-height file.

9 Post-Processing Images (pvPost)

pvPost is the automated ParaView image generator. It reads the converged solution, builds the standard set of geometry, surface-contour, slice, line-integral-convolution (LIC) and iso-surface images, and writes them to postProcessing/images. It runs head-less through pvbatch and is driven entirely by the pvPostSetup file copied into each case, so no manual ParaView session is required.

9.1 Inputs and Outputs

pvPost expects the solution to have been exported to EnSight format. It reads

```
EnSight/<caseName>.case
```

and uses the latest available time step. Iso-surface images additionally read the iso*.vtp files written under postProcessing/surfaces/<latestTime>. All images are written as .jpeg into

```
postProcessing/images/<variable>_<imageType>/
```

one sub-folder per variable and image type. Slice images include the slice normal, position and an index in the file name so they sort in sweep order.

9.2 Running pvPost

pvPost is a ParaView script and must be launched with pvbatch (or pvpython), not the system python, so that the paraview.simple modules are available. From inside the case directory:

```
cd <caseName>
pvbatch --mpi /path/to/postUtilities/pvPost.py >> log.pvpost
```

On the cluster this is wired into the post-processing job through clusterDict and runs automatically after the solve. There is one optional flag:

- -meshOnly - generates only the mesh slice images and skips all field images. Use this to inspect the mesh before a solution exists.

```
pvbatch --mpi /path/to/postUtilities/pvPost.py --meshOnly >> log.pvpostMesh
```

9.3 The pvPostSetup File

caseSetup copies a pvPostSetup file into every case. Edit it to choose which image sets are produced and how the slices are taken. The default file is shown below.

```
[PV_POST_MAIN]
CASE_NAME = default
CAMERA_VIEWS = default
VARIABLE_DICT = default
WHEEL_BASE = default
BACKGROUND_COLOR = default
RESOLUTION = default
GEOM_IMG = True
SURFACE_IMG = True
SLICE_IMG = True
SLICE_LIC_IMG = True
ISO_SURFACE_IMG = True
CREATE_SCALAR_BAR = True

[GEOM]
VIEWS = default

[SURFACE]
VIEWS = default
VARIABLES = default

[SLICE]
VIEWS = default
NORMALS = default
NSLICES = default
SLICE_RANGE = default
VARIABLES = default
LIC_VARIABLES = default

[ISO_SURFACE]
VIEWS = default
```

```
VARIABLES = default
ISO_VALUES = default

[VARIABLES]
SURFACE_VARIABLES = default
SLICE_VARIABLES = default
ISO_VARIABLES = default
```

9.3.1 [PV_POST_MAIN]

- CASE_NAME [str] - case name used in image titles and file names, default uses the case directory name (default, or str)
- CAMERA_VIEWS [str] - path to a camera-views file, default uses the bundled view definitions (default, or a file path)
- VARIABLE_DICT [str] - path to the variable dictionary that defines the plotted variables, their equations, labels, ranges and colour maps, default uses postUtilities/default/defaultVariables (default, or a file path)
- RESOLUTION [str] - working render resolution, default uses the built-in 16:9 setting (default)
- GEOM_IMG, SURFACE_IMG, SLICE_IMG, SLICE_LIC_IMG, ISO_SURFACE_IMG [bool] - master switches for each image set, set to False to skip that set (True, False)
- CREATE_SCALAR_BAR [bool] - draw the colour-bar legend on field images (True, False)

9.3.2 [SLICE]

The slice block controls both the field slices and the LIC slices. Each of VIEWS, NORMALS, NSLICES and SLICE_RANGE is a list, and the lists must all be the same length: entry i defines one slice plane sweep.

- VIEWS [str] - camera view used to render each plane (default = Front Left LeftForward LeftBack Top)
- NORMALS [str] - slice plane normal for each sweep (default = X Y Y Y Z)
- NSLICES [int] - number of evenly spaced slices in each sweep, default adapts to the symmetry of the case
- SLICE_RANGE [[float,m] [float,m]] - start and end coordinate of each sweep as [min,max], default derives sensible ranges from the front-axle reference and the reference dimensions
- VARIABLES [str] - field-slice variables to plot, default plots the standard pressure, velocity and vorticity set
- LIC_VARIABLES [str] - variables coloured underneath the LIC streamlines, default uses the standard set

A variable requested here that is not defined in the variable dictionary, or not present in the solution, is reported and skipped rather than stopping the run.

9.3.3 [SURFACE] and [ISO_SURFACE]

- [SURFACE] VIEWS / VARIABLES - camera views and surface-contour variables painted on the car body (e.g. CpMean, wall shear).
- [ISO_SURFACE] VIEWS / VARIABLES / ISO_VALUES - camera views for the iso-surface renders. The iso-surfaces themselves (isoQ, isoCtp, ...) are produced by the solver's surface function objects and read from postProcessing/surfaces; pvPost renders whatever iso*.vtp files it finds.

9.4 Variables and Camera Views

The variable dictionary defines every plottable variable as an equation over the OpenFOAM fields, together with a display label, a value range and a ParaView colour preset. The tokens UREF, LREF and WREF in the equations are substituted with the case reference values before evaluation, so coefficients such as CpMean and CptMean are normalised consistently with the rest of the case. To use a custom set, point VARIABLE_DICT (and, if needed, CAMERA_VIEWS) at your own file.

To customise one case, copy the default variable and view files from postUtilities/default/ to new files, then reference those files from the case's pvPostSetup. Do not modify the repository defaults unless the change should apply to every future case. For example:

```
[PV_POST_MAIN]
VARIABLE_DICT = myVariables
CAMERA_VIEWS = myViews

[SURFACE]
VIEWS = DriverSide FrontLeft
VARIABLES = CpMean UMeanStreamwise CfMean

[SLICE]
VIEWS = DriverSide
VARIABLES = CpMean UMeanStreamwise
LIC_VARIABLES = UMean
```

Each custom variable file contains the three top-level groups surfaceVariables, sliceVariables and isoVariables. Add a variable to the required groups using an equation, label, display range and colour map. The following entry adds a normalised streamwise mean velocity variable:

```
"UMeanStreamwise": {
    "equation": "UMean_X/UREF",
    "label": "Umean,x/Uref",
    "range": [0, 1.2],
    "color": "Viridis (matplotlib)"
}
```

The name UMeanStreamwise is used in the VARIABLES lists. The equation must reference fields present in the exported EnSight solution. A requested variable that is missing from the dictionary or solution is reported and skipped.

Custom camera-view files use one JSON object per view. The camera position is campos, the point being viewed is focalpos, viewup defines the upward direction, and parallelscale controls the orthographic zoom. Reference tokens can be used in the expressions:

```
"DriverSide": {
    "pp": 1,
    "campos": "[0,-LREF*10,0]",
    "focalpos": "[1.18,0,0.85]",
    "viewup": "[0,0,1]",
    "parallelscale": "WREF/2"
}
```

The name in each VIEWS list must exactly match a key in the custom view file. When CAMERA_VIEWS = default, built-in views are generated automatically from the case reference dimensions.

9.5 Front- and Rear-Wing Pressure Sections

pvPost can extract the mean pressure coefficient where y-normal planes intersect the front or rear wing surface. Enable either section in the case's pvPostSetup file:

```
[FRONT_WING_PRESSURE]
ENABLE = True
PATCH_PATTERN = frontWing.*
NORMAL = default
Y_RANGE = default
N_PLANES = 11
VARIABLE = CpMean
OUTPUT_DIR = postProcessing/wingPressure

[REAR_WING_PRESSURE]
ENABLE = True
PATCH_PATTERN = rearWing.*
NORMAL = default
Y_RANGE = default
N_PLANES = 11
VARIABLE = CpMean
OUTPUT_DIR = postProcessing/wingPressure
```

The default patch patterns are case-insensitive regular expressions. The default NORMAL is the SAE y direction, (0 1 0). Y_RANGE = default samples from CREF_y - 0.5 WREF to CREF_y + 0.5 WREF; an explicit range can be supplied as two values, for example [-0.4,0.4]. N_PLANES controls the number of evenly spaced planes. A different three-component plane normal can be supplied instead of default.

Each intersection is written as a CSV file containing x, y, z and CpMean columns under OUTPUT_DIR. Both sections are disabled by default. This feature writes CSV data only and does not alter the existing surface, slice or LIC image generation.

For a generated cluster case, enable the required sections in the case copy of pvPostSetup before submitting the post-processing job. The standard postPro workflow runs pvPost.py first, creating the CSV files when the sections are enabled, and then runs postRun.py -wingPressure to create the plots. No additional cluster setting is required beyond enabling the relevant section; the command is included in the post section of the selected clusterDict template.

For local or manual processing, run postRun.py after pvPost.py has completed. The selected cases are supplied with -trial:

```
python postUtilities/postRun.py --wingPressure \
    --trial case001 case002 --saveFormat png
```

For each selected case, postRun.py reads the available wing CSV files, plots CpMean against x for each y plane, and writes frontWing_CpMean_vs_x.png and rearWing_CpMean_vs_x.png under postProcessing/wingPressure. The pressure-coefficient axis is inverted using the conventional aerodynamic presentation. Cases without enabled wing extraction or matching CSV files are reported and skipped.

9.6 Half-Symmetry Cases

When the case is built on half symmetry (SIM_SYM = half), pvPost automatically reflects the body surface, the internal volume and the iso-surfaces about the symmetry plane so the rendered images show the full car.

9.7 Cornering (SRF) Cases

For an SRF cornering case the time-averaged velocity stored in the solution is the rotating-frame relative velocity UrelMean, not the absolute UMean. pvPost detects a cornering case automatically from CORNERING_SETUP -> RUN_CORNERING and, when it is True:

- rewrites every variable equation that references UMean (including its components UMean_X/Y/Z and the UMean^2 term in CptMean) to read UrelMean instead, and
- uses UrelMean as the vector field for the LIC streamlines.

Variable names, labels and ranges are unchanged, so the same pvPostSetup works for both straight-line and cornering runs and the image file names stay consistent. A line is printed to log.pvpost confirming the substitution:

```
Cornering (SRF) case detected: using UrelMean in place of UMean.
```

NOTE: This requires UrelMean to be present in the solution. The cornering setup averages both U and Urel, so UrelMean is available; if it is missing, those variables are reported as unavailable and skipped.

9.8 Performance

For speed, pvPost loads only the fields actually referenced by the active variable equations (plus the LIC vector) from the EnSight reader, dropping unused fields such as phi, k, omega, nut and the Prime2Mean tensors before the cell-to-point conversion and slicing. It also reuses a single slice filter and calculator per slice normal, updating only the slice position as it sweeps, rather than rebuilding the pipeline for every slice. These changes reduce runtime and memory with no change to the images produced. The console reports which fields were loaded and which were skipped:

```
Loading only required fields: ['pMean', 'UMean', ...]
Skipping unused fields: ['phi', 'k', 'omega', ...]
```

9.9 Workflow Summary

- Export the converged solution to EnSight (`EnSight/<caseName>.case`).
- Edit `pvPostSetup` to choose the image sets (`GEOM_IMG`, `SURFACE_IMG`, `SLICE_IMG`, `SLICE_LIC_IMG`, `ISO_SURFACE_IMG`) and, if needed, the slice `NORMALS`, `NSLICES` and `SLICE_RANGE`.
- Run `pvPost` with `pvbatch` from the case directory (automatic on the cluster). Add `-meshOnly` to render only the mesh.
- Collect the images from `postProcessing/images`. Cornering cases are handled automatically; no setup change is required.

9.10 Troubleshooting

- `ERROR! Unable to find EnSight File!` - no `EnSight/<caseName>.case` was found. Export the solution to EnSight before running `pvPost`.
- `The inputs in pvPostSetup is not correct for [SLICE]` - the `VIEWS`, `NORMALS`, `NSLICES` and `SLICE_RANGE` lists are not all the same length. Make every `[SLICE]` list have one entry per sweep.
- `<variable> not available, skipping` - the variable is not defined in the variable dictionary or not present in the solution. Check the spelling in `pvPostSetup` and confirm the field was written by the solver.
- `Could not determine required fields from variables, loading all` - field detection found nothing to load and fell back to loading every field. The run still completes; check that the variable dictionary equations reference real field names.
- `No iso-surface files found, skipping` - no `iso*.vtp` files exist under `postProcessing/surfaces`. Enable the iso-surface function objects in the solver setup if iso images are required.

10 Case Summaries (postRun.py –summary)

`postRun.py -summary` reads the converged solver logs and function-object output for a case and writes a single `summary.csv` file at the case root. `summary.csv` is a two column, header-less file of `key,value` pairs, one row per reported quantity (force and moment coefficients, cell count, mesher, solve time, run date, and so on). Every numeric value is formatted to three decimal places before being written, so downstream tools and spreadsheets show a consistent precision regardless of the underlying solver output.

```
python postUtilities/postRun.py --summary
```

10.1 Case Completeness

Before summarising, `postRun.py` checks that the case actually finished by looking for a standalone `End` line in the relevant solver log. It checks, in order, `log.simpleFoam`, `log.pisoFoam`, `log.SRFSimpleFoam` and `log.SRFPimpleFoam`, so both straight-line and cornering (SRF) cases are recognised as complete. A case without a matching, finished log is reported as incomplete and skipped wherever a completeness check is required (single-case summaries, ride-height parent averaging and sensitivity plotting).

10.2 Ride-Height Parent Summaries

When `-summary` is run inside a ride-height mapping parent case (a directory containing child cases named `<parentCaseName>\_<number>`), `postRun.py` does not recompute coefficients from the raw logs. Instead it reads each child's own `summary.csv`, skips any child that is incomplete, missing its `summary.csv`, or has an unreadable `summary.csv` (with a warning printed for each skipped child), and averages the remaining numeric fields into the parent's own `summary.csv`. This includes the lift-balance and side-force coefficients `Cl(f)`, `Cl(r)`, `Cs(f)` and `Cs(r)` alongside `Cd` and `Cl`.

11 Ride-Height Sensitivity Plots (postRun.py –sensitivityPlots)

For a ride-height mapping parent case, `postRun.py -sensitivityPlots` detects and plots single-variable sensitivity sweeps directly from the parent's `rideHeights\_updated.csv` and each child's `summary.csv`. It only runs for parent cases (those with child directories matching `caseName\_\#`); it does nothing for a plain single case or when run from inside a child case.

```
python postUtilities/postRun.py --sensitivityPlots
python postUtilities/postRun.py --sensitivityPlots --includeSideForce
python postUtilities/postRun.py --sensitivityPlots \
    --sensitivityCases ../otherRideHeightMap ../anotherRideHeightMap
```

11.1 Sweep Detection

The ride-height map columns are organised into groups: `Front Ride Height` (`fl`, `fr`), `Rear Ride Height` (`rl`, `rr`), `Yaw` (`yaw`) and `Corner` (`corner\_radius`, `corner\_dir`). The `steer` column (or `steer\_deg`/`steer\_angle`) is tracked separately as a constancy-only group: it is never itself plotted, but it must stay constant for a group to qualify as a clean single-variable sweep. Within a group, columns that move together (for example `fl` and `fr` changing in step) are averaged into one representative value, so both changing together still counts as a single variable rather than two.

Because a ride-height map is often a full grid (front ride height crossed with rear ride height, for example), a group can produce several sweeps at different fixed values of everything else. `postRun.py` finds every fixed-combo subset of the child cases where all other groups (including `steer`) hold one constant value, and where the named group still varies across at least three of those rows; each qualifying subset is plotted as its own sweep. For example, a map that varies both front and rear ride height produces one Front Ride Height sweep for each rear ride height value that has enough points, and vice versa.

11.2 Sweep Plots

Each sweep is drawn as one figure with one subplot per force coefficient: `Cd`, `Cl`, `Cl(f)` and `Cl(r)` by default, with `Cs(f)` and `Cs(r)` added when `-includeSideForce` is given. Each subplot shows the raw data points plus a low-order polynomial curve fit (degree scales with the number of available points, capped at three, and falls back to a straight line when there are too few unique x-values to fit safely). The figure title names only the swept variable; the values held constant for that sweep (the other groups and `steer`) are listed in a boxed side panel instead, so long configuration lists do not overrun the plot. Each figure is saved to

```
postProcessing/sensitivityPlots/<SweepName>_sweep.png
```

with a suffix built from the fixed configuration values when more than one sweep of the same type exists (for example `FrontRideHeight\_rl-0.01\_sweep.png`).

11.3 Comparing Multiple Ride-Height Maps

The optional `-sensitivityCases` flag takes one or more other ride-height mapping parent case paths. For each of those cases, `postRun.py` builds its own sweeps in the same way, then overlays a sweep from another case onto the primary case's matching sweep only when both the sweep type

and every fixed configuration value are identical (same held rear ride height, yaw, corner and steer, for example). Matching sweeps are drawn in the same figure with one colour per case and a legend; sweeps that do not have a matching fixed configuration in another case are left as single-case plots. This makes it possible to compare, for example, a Front Ride Height sensitivity between two different aero concepts at the same rear ride height.

11.4 Front-Rear Ride Height Contour Plots

When the map contains a rectangular grid of front and rear ride height values with roll held at zero, `postRun.py` additionally produces a Front-Ride-Height-vs-Rear-Ride-Height contour plot for each qualifying fixed combination of the remaining variables (Yaw, Corner and steer). The contour requires at least four grid points with at least two unique front and two unique rear values. Each force coefficient is interpolated over the front/rear grid using cubic interpolation (`scipy.interpolate.griddata`, `method='cubic'`) and rendered as a filled contour with contour lines and the underlying data points overlaid, arranged two-by-two across the default four metrics (six when `-includeSideForce` is set, with any unused subplot slots hidden). Output is saved to

```
postProcessing/sensitivityPlots/FrontRearRideHeight_contour<suffix>.png
```

Contour plots are always single-case; they are not affected by `-sensitivityCases`.

12 Pushing Results to Google Sheets (gsheetExport.py)

`postUtilities/gsheetExport.py` pushes a case's summary metrics to a Google Sheet, one row per trial, using a service account for authentication. It reads its values directly from the case's `summary.csv` rather than recomputing anything from the solver output, so it must be run after `postRun.py -summary` has produced an up to date `summary.csv`.

When the case being pushed is itself a child of a ride-height mapping parent (its directory name matches `<parentCaseName>\_<number>`), `gsheetExport.py` first regenerates the parent's `summary.csv` by invoking `postRun.py -summary` in the parent directory, then pushes both the child's row and the parent's (map-averaged) row to the sheet. For a case that is not part of a ride-height map, only that case's own row is pushed. The fixed set of spreadsheet columns written for each row includes run date, solve time, cell count, mesher, half/full symmetry, velocity, yaw, density, reference area and wheelbase, `Cd`, `Cl`, `Cl(f)`, `Cl(r)`, `Cs(f)`, `Cs(r)`, confidence intervals for `Cd`/`Cl`, and the front and rear wing `Cd`/`Cl`.